/ MMB8052 - Practical 4

MMB8052 Practical 04 - Introduction to R Programming

Introduction

R is an open-source, free programming language created by statisticians Ross Ihaka and Robert Gentleman in the 1990s. R was intended as a language for statistical analysis and data visualisation. It is used extensively in bioinformatics because it enables the manipulation and analysis of very large data sets.

R is an extensible environment and new functionality can be added to the core language via the installation of *packages*. There are large existing repositories of these packages for R, including the Comprehensive R Archive Network (CRAN) which includes thousands of packages covering many use-cases, and Bioconductor which caters specifically for the analysis of biological data. Many packages published in these repositories are peer-reviewed products of statistical and bioinformatics research.

These key features of R (open source, extensible, designed for statistics) mean that it has found a niche as one of the principle programming languages for *data science*, along with Python. We are focussing on R largely because of the Bioconductor repository of bioinformatics packages, which is an invaluable resource for biological data analysis.

RStudio

In these practicals we will be using RStudio as an interface to R. RStudio is an *Integrated Development Environment* (IDE) designed specifically for working with R. It is a relatively user friendly interface to R, and has many advantages over using the stand-alone R command prompt. R and RStudio are installed on the computers in the clusters we are using for these practicals - look for RStudio in the Start menu.

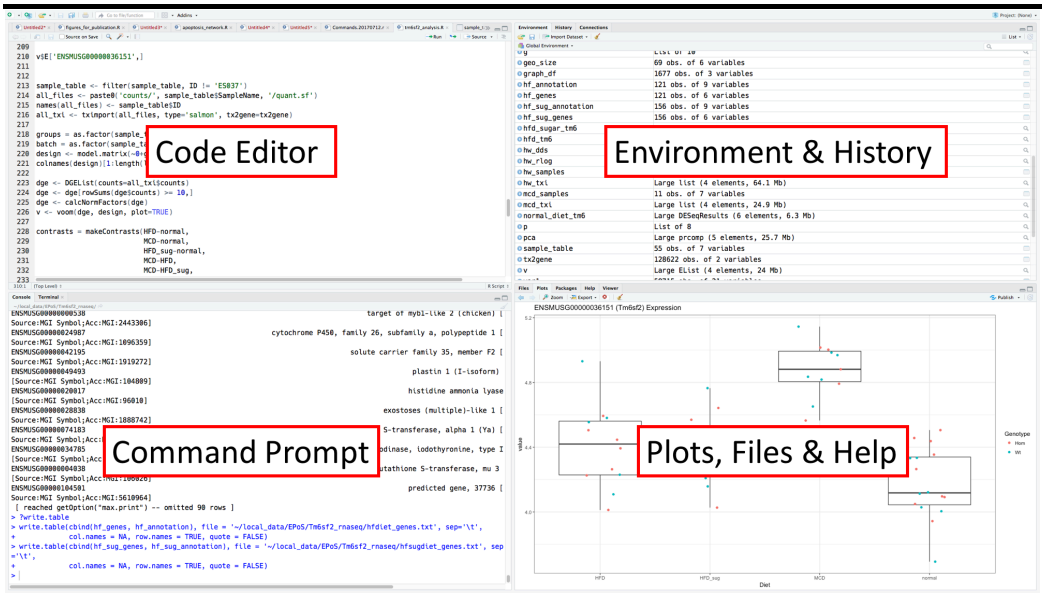


Figure 1: The typical 4 pane layout of RStudio

RStudio splits the application window into up to 4 panes, by default these panes contain a code editor (top left, closed when you first run RStudio), the R command prompt (also known as the *console*, bottom left), the current R *environment* and a command history (top right) and plots, a file explorer, and help information (bottom right). These panes are all inter-related, for example, code written in the code editor can be sent to the console by pressing Ctrl+Enter. The results of executing this code can also effect the other panes - new variables created will appear in the Environment, graphical output will appear in Plots, and so on.

Installing R and RStudio

If you want to work on your own computer, you will need to install R and RStudio:

- Download & install the correct version of R (https://www.stats.bris.ac.uk/R/) for your operating system
- Download & install the correct version of RStudio (https://www.rstudio.com/products/rstudio/download/#download) for your operating system

Exercise 4.1

Estimated time: 5 minutes

- Open RStudio, and type the following mathematical commands in the Console
    - Each command starts with a `>` - this is to show where the command begins (c.f. the `$` we used for bash commands) - you don't need to type this

```
> 4 + 9
> 2 + 5 * 7
> (2 + 5) * 7
> 10.2 / 3 - 2
> 10.2 / (3 - 2)
> 2 ^ 4
> 7 / 3
> 7 %/% 3
> 7 %% 3
```

- From the results of this operations, can you figure out:
    - How R handles the order of mathematical operations

- What the `^` symbol does
- What the 3 different division related (`/`, `%/%` and `%%`) operators are doing

Data Types (single values)

Every value in R has a *data type*. There are 5 basic data types:

- character (e.g. `"a"`, `"bioinformatics"`)
- double (decimal numbers, e.g. `2`, `2.1`, `pi`)
- integer (whole numbers, e.g. `2L`)
- logical (`TRUE`, `FALSE`)
- complex (e.g. 3 + 2i)

Integer and double types are collectively known as *numerics*. We are only going to encounter characters, numerics and logicals during this course.

Operators

Operators are symbolic representations of mathematical, relational and logical manipulations. R has a range of operators to represent a number of these concepts. In exercise 4.1 we used a range of mathematical operators:

Operator	Description
+	Addition
-	Subtraction

Operator	Description
*	Multiplication
/	Division
^	Exponent
%/%	Integer division
%%	Modulus (remainder from integer division)

What about the equals sign (=)?

The one major mathematical symbol notable by its absence here is the equals sign. In R (as in many other programming languages) `=` is used for the *assignment* of a value to a variable. We've already encountered variable assignment in `bash`, and R works in a similar way (but with a simpler syntax). The result of the operation on the right of `=` is assigned to the variable name on the left:

```
> x = 2 + 2
> x
[1] 4
```

We'll come back to what the `[1]` means later, the result here is showing you that `x` has the value `4` (i.e. the result of `2 + 2`).

Relational operators

This class of operator is used to compare between values. The way this works is most obvious with numerical values, but all data types in R have a natural order. The output from the use of a relational operator is a logical (`TRUE` or `FALSE`).

Operator	Description
<	Less than
>	Greater than
<=	Less than or equal to
>=	Greater than or equal to
==	Equal to
!=	Not equal to

```
> 1 > 2
[1] FALSE
> 1 < 2
[1] TRUE
> 6 + 2 == 4 + 4
[1] TRUE
> "a" < "b"
[1] TRUE
> TRUE < FALSE
[1] FALSE
```

Exercise 4.2

Estimated time: 5 minutes

Use relational operators to work out the answers to the following questions:

- We've seen already that `"a" < "b"`, but are capital letters higher, lower or equivalent to their lowercase counterparts in R?
- Using R, show that `TRUE == 1` - do you know *why* this is the case?
- Given the above, what numerical value is equivalent to `FALSE`?

Logical Operators

Logical operators compare boolean values with a set of standard logic rules. This functionality allows us to compare the results of comparisons made with relational operators.

Operator	Description
`&`	AND
`|`	OR
`!`	NOT

Truth tables can be used to show how these operators combine boolean values.

AND operator

A	B	A & B
FALSE	FALSE	FALSE
TRUE	FALSE	FALSE
FALSE	TRUE	FALSE
TRUE	TRUE	TRUE

OR operator

A	B	A I B
FALSE	FALSE	FALSE
TRUE	FALSE	TRUE
FALSE	TRUE	TRUE
TRUE	TRUE	TRUE

NOT operator

A	!A
TRUE	FALSE

A	!A
FALSE	TRUE

We can also demonstrate their use in practice:

```
> x = 12
> x > 10 & x < 15
[1] TRUE
```

In the above, the first part (`x > 10`) will evaluate to `TRUE` - since `x` was assigned the value of `12`. The second part (`x < 15`) also evaluates to `TRUE`. Since with a logical AND (`&`) both parts need to be `TRUE` for the expression as a whole to be `TRUE`, this statement evaluates to `TRUE`.

Exercise 4.3

Estimated time: 5 minutes

`x = 5` and `y = 12`. Work out the boolean results of the following statements. Use R to test and see if you were correct.

- `x > 6 & y < 16`
- `x < 6 & y < 5 * 3`
- `x > y / 4 | y > 2006 / 167`
- `!x == 5 | !y == 11`

Functions

R includes many *functions*, which can be considered as sub-programs which take inputs (arguments), carry out calculations and return outputs. In the following example, the logarithm function `log` is being called by entering its name, followed by round brackets. This returns the natural logarithm of the argument (in this case 10, which is included inside the brackets):

```
> log(10)
[1] 2.302585
```

The behaviour of this function can be further modified by providing additional optional *arguments*. In this case, we can specify the base to calculate the logarithm in by providing the named `base` argument. Arguments to functions are included in the brackets, and are separated by commas:

```
> log(10, base=10)
[1] 1
```

R functions have documentation (like `man` pages in Linux). The way to access the help for a function is to precede the function name with `?`. So to view the help page for `log()`, type `?log` in the console. This help page describes the purpose of the function, and all of the arguments which it understands.

Vectors

So far, we have considered single values as variables but R is designed to work with sets of values. A one-dimensional set of values in R is called a *vector*, and can be created using the `c()` function.

```
# Vector of numerics
> c(0, 1, 1, 2, 3, 5, 8, 13, 21)
[1]  0  1  1  2  3  5  8 13 21
# Vector of characters
> c("a", "b", "c", "d")
[1] "a" "b" "c" "d"
# Vector of booleans
> c(TRUE, FALSE, TRUE)
[1] TRUE FALSE TRUE
```

All elements of an R vector must be the same data type. If you create a vector which violates this rule, R will silently convert the vector to a single data type. The data type of the output vector will be that of the highest type of the input elements according to the hierarchy logical < double < character. See the following examples:

```
# A boolean vector which contains a numeric becomes
> c(TRUE, FALSE, 3)
[1] 1 0 3
# A numeric vector which contains a character become
> c(1, 2, 3, "X", "Y", "M")
[1] "1" "2" "3" "X" "Y" "M"
# Any vector which contains a character is made into
> c(1, 2, 3, TRUE, "Hello", "World")
[1] "1"     "2"     "3"     "TRUE"  "Hello" "World"
```

We can also create numeric vectors using the `seq()` function, or the colon (`:`) operator:

```
> seq(1, 6)
[1] 1 2 3 4 5 6
> 1:6
[1] 1 2 3 4 5 6
```

The kind of operations we looked at with single values above can also be applied to vectors. For example, mathematical operators will act on corresponding elements of equal length vectors:

```
> x = c(2, 4, 6, 8, 10)
> y = c(1, 3, 5, 7, 9)
> x + y
[1]  3  7 11 15 19
> x * y
[1]  2 12 30 56 90
```

Where vectors are unequal lengths, the shorter vector will be *recycled* until it matches the length of the longer vector:

```
> x = c(2, 4, 6, 8, 10, 12)
> y = c(10, 100)
> x * y
[1]   20  400   60  800  100 1200
```

In this example, the recycling of `y` makes it `c(10, 100, 10, 100, 10, 100)`.

Relational operators used with vectors will give back a boolean vector of the same length as the input, with a result for each element of the input vector:

```
> x = 1:10
> x >= 5
[1] FALSE FALSE FALSE FALSE TRUE  TRUE  TRUE  TRUE
```

Filtering vectors

It is often the case that we want to filter a vector and keep only elements which match certain criteria. This can be done using square brackets `[]` - this is an example of an *extract* operator. You can get more information by typing `?'['` in the console. Extraction works with numerical indices, names or boolean vectors.

Index filtering

```
> gene_id <- c("ENSG00000030110", "ENSG00000166394",
> gene_id[2]
[1] "ENSG00000166394"
> gene_id[c(1,3)]
[1] "ENSG00000030110" "ENSG00000109738"
```

Name Filtering

It is possible to add *names* to a vector, to give each element a character-based reference, as well as a numeric one. These names can then be used to select specific elements.

```
> names(gene_id) = c("BAK1", "CYB5R2", "GLRB", "MERT
> gene_id["CYB5R2"]
          CYB5R2
"ENSG00000166394"
> gene_id[c("MERTK", "CYB5R2")]
            MERTK            CYB5R2
"ENSG00000153208" "ENSG00000166394"
```

Boolean Filtering

Probably most usefully, we can extract elements using a vector of logicals. R will give you back only the elements corresponding to `TRUE` in the boolean vector. We can combine this method with the relational operators from before, to give us a powerful system for filtering (even very large) datasets.

```
> bools = c(TRUE, FALSE, TRUE, FALSE)
> gene_id[bools]
```

```
              BAK1               GLRB
"ENSG00000030110" "ENSG00000109738"
> gene_id == "ENSG00000109738"
 BAK1 CYB5R2   GLRB  MERTK
FALSE  FALSE   TRUE  FALSE
> gene_id[gene_id == "ENSG00000109738"]
> x = 1:10
> x[x>=5]
[1]  5  6  7  8  9 10
```

Exercise 4.4

Estimated time: 10 minutes

R contains lots of functions for mathematical operations. Read the help for the following functions:

- `mean`
- `sd`
- `range`
- `summary`
- What do they do?
- Make a vector of a thousand random values from a normal distribution: `x = rnorm(1000)`
- What are the results of the above functions when applied to these values?
    - Are these results what you'd expect, given the function of `rnorm`?
    - Compare results with your neighbours.

- Filter `x`, keeping only observations greater than 1.
    - How many are you left with? (HINT: `length()`)
    - How does this compare with your neighbours?
- Finally, produce a visual summary of your random data by running `hist(x)`.

Two-dimensional Data

Matrix

A matrix is a vector with two additional attributes - the number of rows (`nrow`) and the number of columns (`ncol`) which define the shape of a two-dimensional data structure. You can create a matrix using the `matrix()` function. Note that matrices are constructed in column-major order (i.e. the values in the vector used to construct the matrix fill it up column by column); you can request row-major ordering by setting the `byrow` argument of the matrix function to `TRUE`.

Like a vector, all of the elements in a matrix have to be of the same type, and on creation will be caste to the highest type included.

```
> matrix(1:12, nrow=4, ncol=3)
     [,1] [,2] [,3]
[1,]    1    5    9
[2,]    2    6   10
[3,]    3    7   11
[4,]    4    8   12
> matrix(1:12, nrow=4, ncol=3, byrow=TRUE)
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
```

```
[3,]    7    8    9
[4,]   10   11   12
```

We can extract from matrices using square brackets as with vectors, but since a matrix is a 2-dimensional structure, we supply two values separated by a comma `[ , ]`. The first value (on the left of the comma) extracts rows of the matrix, and the second value (on the right of the comma) extracts columns of the matrix (so the format is `[rows, columns]`).

```
> my_matrix = matrix(1:12, nrow=4, ncol=3)
> my_matrix[3:4, 1:2]
     [,1] [,2]
[1,]    3    7
[2,]    4    8
```

Exercise 4.5

Estimated time: 5 minutes

Work out what the following functions do to the matrix `my_matrix` (as per the code above).

- `t()`
- `dim()`
- `as.vector()`

Data Frame

A data frame has a 2-dimensional rows-and-columns structure like a matrix. However, unlike a matrix each column can be of a different data type (for example some columns might contain numbers and other columns might contain character strings). This makes data frames particularly useful for storing tabular data in which the rows correspond to cases and columns correspond to variables. Data frames are the fundamental data structure used by R's statistical modelling software.

The `data.frame()` function creates a data frame from a set of equal length vectors:

```
> df = data.frame(gene_id = c("ENSG00000030110", "EN
                  gene_symbol = c("BAK1", "CYB5R2", "G
                  logFC = c(0.28, -1.36, -0.89, 0.77),
                  pVal = c(0.73, 0.004, 0.08, 0.02))
> df
          gene_id gene_symbol logFC  pVal
1 ENSG00000030110        BAK1  0.28 0.730
2 ENSG00000166394      CYB5R2 -1.36 0.004
3 ENSG00000109738        GLRB -0.89 0.080
4 ENSG00000153208       MERTK  0.77 0.020
```

We can access the columns of a data frame as vectors using the special `$` operator:

```
> df$gene_symbol
[1] "BAK1"   "CYB5R2" "GLRB"   "MERTK"
> df$pVal
[1] 0.730 0.004 0.080 0.020
```

We can also add new information into a data frame using this notation:

```
> df$new_column = c("up", "down", "down", "up")
> df
          gene_id gene_symbol logFC  pVal new_column
```

```
1 ENSG00000030110         BAK1  0.28 0.730         up
2 ENSG00000166394       CYB5R2 -1.36 0.004       down
3 ENSG00000109738         GLRB -0.89 0.080       down
4 ENSG00000153208        MERTK  0.77 0.020         up
```

Filtering of a data frame works in the same way as a matrix, using `[rows, columns]`.

```
> df[2, 2]
[1] "CYB5R2"
> df[df$gene_symbol == "BAK1",]
          gene_id gene_symbol logFC pVal
1 ENSG00000030110         BAK1  0.28 0.73
```

In this last example, the expression `df$gene_symbol == "BAK1"` which is inside the square brackets, tests the `gene_symbol` column as though it were a vector, and returns a logical vector with a result for each element (so will return `c(TRUE, FALSE, FALSE, FALSE)`). This vector has the same number of elements as the data frame has rows, so will return only the rows of the data frame which meet the condition.

Exercise 4.6

Estimated time: 5 minutes

Using the above information, can you filter the data frame for rows which are "significant" (according to the p-values recorded)?

Can you work out how to add this information in to the data frame without filtering it?

Summary

This practical has introduced the basics of the R programming language. We've seen the types of data we can use in R, and how to manipulate that data using elemental operators and base functions. Practical 5 will be an introduction to some more advanced R programming, with a particular focus on the visualisation of data.